

MRI Interlock System

Overview

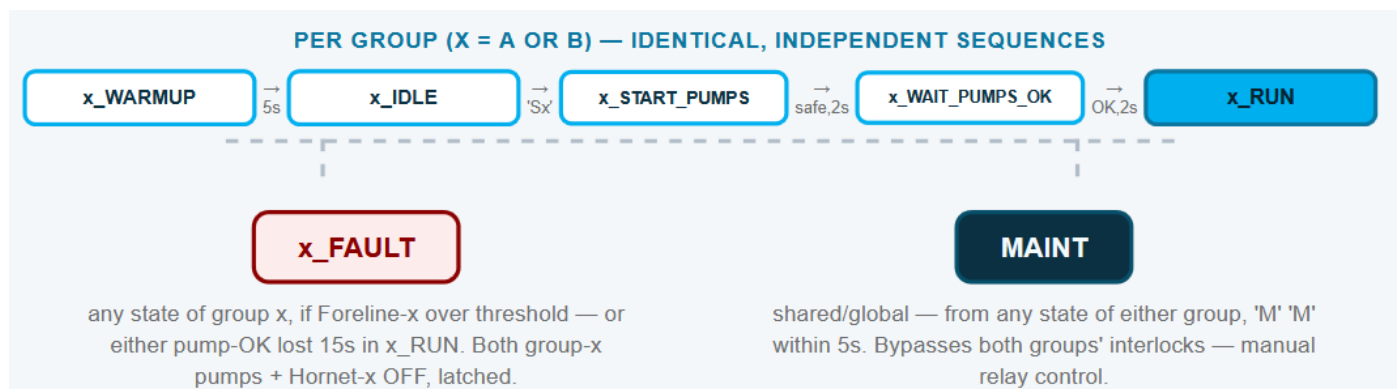
An **Arduino Mega 2560** runs the safety logic in firmware (`MRI_INT_SYS`). A Python/PyQt5 desktop app connects to it over USB serial to show live pressures, control the pumps, and log data. **All safety logic lives in the firmware, not in Python** — the app is a passive display/ control client, and losing the serial connection does not shut the system down. This version has been updated to work with twice the number of pumps and gauges, and has split these additional components into **two independent vacuum subsystems** — Group A and Group B — sharing one Arduino chassis. A fault, start, stop, or reset on one group has zero effect on the other. The only shared/global piece of state is Maintenance mode.

Components	Group A (850i Pumps)	Group B (350i Pumps)
Turbopumps	TV850i-A, TV850i-B	TV350i-A, TV350i-B
Stinger Foreline gauge	Foreline-A	Foreline-B
Hornet Chamber gauge	Hornet/UHV-A	Hornet/UHV-B

Each pump has its own pump-OK feedback signal; each Hornet gauge is relay-controlled and can switch off independent of its group's pumps; each Foreline gauge is always on, no relay. The Arduino reads all 4 gauges and all 4 pump-OK signals, runs both groups' interlocks, and reports everything once a second as one CSV line covering both groups.

Firmware Behavior

The firmware runs two fully independent state machines — two instances of the same `StateMachine` class, one per group. Each steps through the same sequence below using only its own gauges, pumps, and relay. Maintenance mode is the one exception — a single shared toggle that bypasses both groups' interlocks at once.



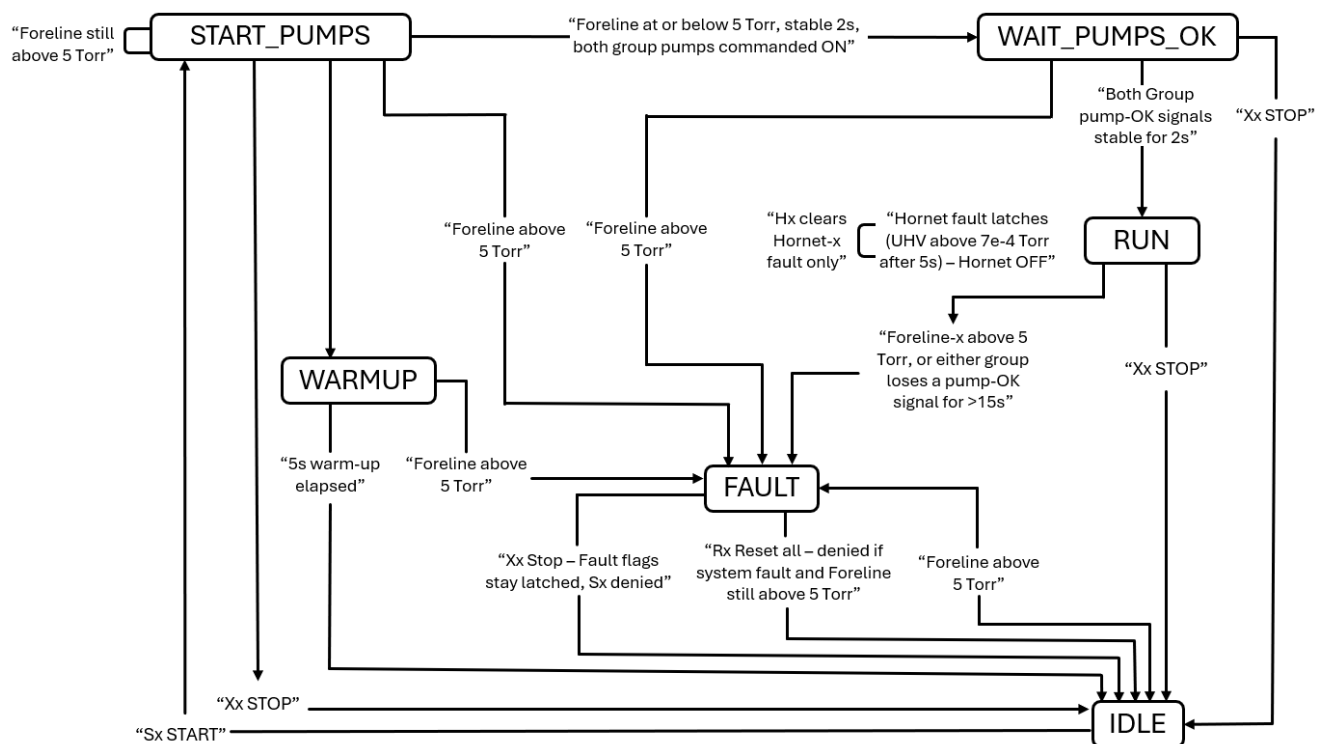
Hornet-x fault

Hornet-x's UHV gauge trips its threshold while group x is in x_RUN. Only that Hornet relay turns off; that group's pumps keep running, the other group is unaffected. Cleared with Hx (or Rx). While not latched, Hornet-x stays energized as long as at least one of that group's two pumps is instantaneously OK.

System-x fault

Foreline-x over threshold (checked in every state of group x), or either of group x's pump-OK signals missing 15s in x_RUN . Both of group x's pumps and Hornet-x turn off. Cleared with Rx , refused while Foreline-x is still unsafe.

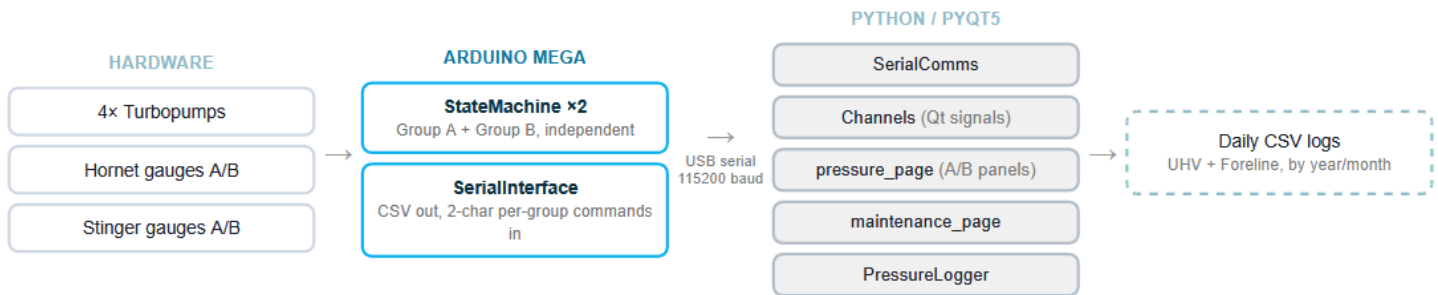
Sending Xx (Stop) from group x's FAULT moves that group's displayed state back to IDLE but does not clear its fault flags — Sx is still refused until Rx or Hx is sent. Each group's state and fault flags persist to EEPROM in a separate address range, so a power-cycle or reconnect never loses one group's running sequence or latched fault — or touches the other group's.



“Note: Fault clearing described in here can be done only in the Arduino IDE. Faults can be cleared in the GUI via the maintenance menu.”

Software Architecture

The GUI never talks to the serial port directly, and the serial/logging layers never touch Qt widgets — everything passes through a signal bus. **pressure_page** renders two side-by-side group panels from the one CSV stream.



SerialComms: owns the serial port, polls it non-blocking, and emits Qt signals, never touches widgets.

Channels: is the signal bus connecting the serial layer to the GUI.

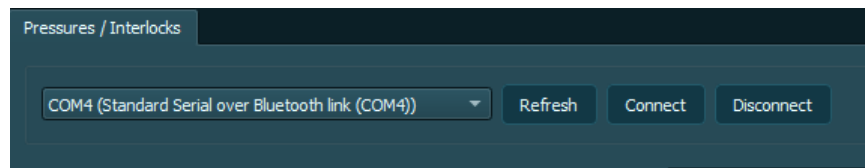
pressure_page: is the main GUI — two side-by-side Group A / Group B panels (each with its own state/fault labels and Startup/Shutdown/Hornet-reset/Reset-all buttons), 4 display-only pump widgets, and a live chart.

maintenance_page: is the manual relay dialog, usable only once firmware confirms MAINT mode — bypasses both groups at once.

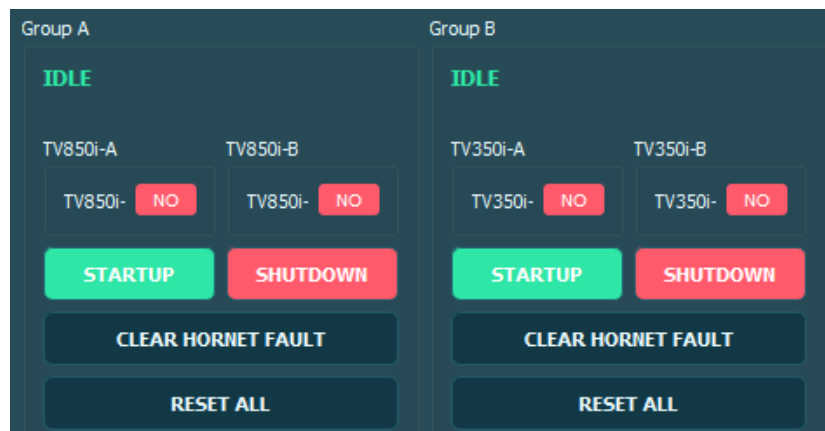
PressureLogger: writes UHV and foreline CSVs daily, rotating at 3:00 AM

Operating the System

1. Select the Arduino's COM port and click **Connect**.



2. Group A and Group B each have their own **STARTUP / SHUTDOWN** and fault-reset buttons — operating one group never affects the other. The 4 pump widgets are display-only.



3. **MAINTENANCE** opens the manual relay dialog — bench testing only since it bypasses every interlock above. If an interlocks tripped, the Maintenance menu can be used to reset the interlocks and allow for a return to normal operation. Only do this if the reason for the tripped interlock is known and has been fixed.

Note: All setpoints (trip thresholds, timing windows) live in firmware only, so the safety logic keeps working even if the GUI is closed. The four thresholds are shared constants evaluated independently against each group's own gauges — there's no way to set a different threshold per group without editing the firmware logic itself. Changing a setpoint means editing the firmware config and re-flashing the Mega — never in Python. See README.md §3 for the checklist before touching a threshold.